

cloud-architecture-lab

Claude Code Tooling Reference — how `.claude/`, `docs/standards/`, and the daily workflow fit together
Generated 2026-06-16 · current state: Day 009 complete, Day 010 next

Contents

- 1. System map — the four layers
- 2. Inside `.claude/` — orchestration layer
 - 2.1 Slash commands
 - 2.2 Skills
 - 2.3 Hooks & settings
 - 2.4 Instructions (the master workflow)
- 3. `docs/standards/` — every file, what's actually in it
- 4. Other key files referenced throughout this document
- 5. The daily workflow, step by step
- 6. A day in the life — worked example
- 7. `.scripts/` — automated guards
- 8. Findings from this audit
- 9. Expected results

1. System map — the four layers

This repo is two things at once: a .NET 8 AI gateway (`src/lab-observability-api/`) and a self-documenting **process** for building it day by day, encoded almost entirely as Claude Code configuration. The process layer has four tiers, each one consuming the layer below it.

1. Instructions

`.claude/instructions/`

The master script.
Defines STEP 0–12 and
what model/surface
runs each one.

2. Commands & Skills

`.claude/commands/`
`.claude/skills/`

The verbs the
instructions invoke. A
command is a one-shot
recipe; a skill is reusable
know-how a command
or you can pull in.

3. Standards

`docs/standards/`

The knowledge
commands/skills read
from and write
decisions into. Naming
rules, error taxonomy,
cert mapping, etc.

4. Artifacts

`docs/notes/` ,
`docs/adr/` , `src/`

What actually gets
produced: code, ADRs,
day notes, the
changelog.

Two enforcement mechanisms run underneath all four layers:

Hooks

`.claude/settings.json`

Fire automatically on every `Write` / `Edit` — no

Scripts

`scripts/*.js`

Stateless checkers (`md-backtick-wrap.js` , `symbol-`

command has to remember to call them.

`drift-check.js`) invoked by hooks, commands, and STEP 8 of the workflow.

2. Inside `.claude/` — orchestration layer

2.1 Slash commands (`.claude/commands/*.md`)

Each file is a prompt template Claude Code expands when you type the matching `/name` . There are 7 registered commands.

Command	Workflow step	What it does	Reads	Writes
<code>/new-day N slug</code>	STEP 3	Scaffolds the empty day skeleton — folder, summary/checklist stubs, architecture doc, Infra template — before any content exists	<code>docs/notes/_index.md</code>	New <code>Day-NNN/</code> folder, <code>changelog.md</code> section, architecture stub, Infra stub
<code>/adr title</code>	STEP 6	Creates the next-numbered ADR from the standard template, Status: Proposed	<code>adr-writer/SKILL.md</code> , existing ADR numbers	New <code>docs/adr/ADR-NNN-*.md</code>
<code>/collab-lens N</code>	STEP 6	Inserts a bounded "who does this serve" block into <code>01-summary.md</code> , phase-filtered against the collaboration map	<code>collaboration-lens/SKILL.md</code> , <code>collaboration-map.md</code> , <code>01-summary.md</code>	<code>01-summary.md</code> (insert), <code>changelog.md</code> (row)
<code>/deploy</code>	STEP 9	Pre-deploy gates (tests + pillars audit) → Kudu zip publish → post-deploy verification	<code>azure-deploy/SKILL.md</code> , <code>Infra/Day-NNN/appsettings-template.md</code>	Azure App Service, <code>06-deployment-log.md</code>
<code>/cert-update N [all]</code>	STEP 10	Reads the day's Certification Reinforcement section, generates concepts/practice-q/resources for one touched domain (or all, with <code>all</code>)	<code>01-summary.md</code> , MS Learn MCP	<code>docs/certifications/<exam>/domains/.../*.md</code>
<code>/cert-scaffold EXAM</code>	one-time per exam	Fetches the official skills outline, builds the domain folder structure — does not generate explanations	MS Learn certification page	<code>docs/certifications/<EXAM>/</code> structure
<code>/repo-audit N</code>	STEP 12	9-check end-of-day health scan: folder completeness, ADR structure, lint, backticks, <code>CLAUDE.md</code>	Everything	Auto-fixes lint/backticks; reports the rest

accuracy,
changelog
coverage, cert
coverage, provider
abstraction,
symbol drift

Not a real command: `/devil`, `/pitch`, and `/10x` are referenced in `daily-workflow.md` 's command table but are *not* files in `.claude/commands/`. They are "paste this exactly" prompt blocks embedded directly in `daily-workflow-steps.md` (STEP 4, 5, and 11) — reasoning gates you run inline in a chat, not slash commands the harness resolves. Active from Day 010+.

2.2 Skills (`.claude/skills/*/SKILL.md`)

Skills are reusable knowledge packs. Unlike commands, they're not typed directly — they're invoked by name from inside a command's instructions, or pulled in by Claude Code automatically when the situation matches their description.

Skill	Invoked by	What it knows
<code>adr-writer</code>	<code>/adr</code>	ADR template, heading structure, "verb-led title", at least 2 alternatives with rejection reasons
<code>azure-deploy</code>	<code>/deploy</code>	The Kudu zip path and why <code>az webapp deploy</code> fails on this network; SSL workarounds; post-deploy verification checklist
<code>collaboration-lens</code>	<code>/collab-lens</code>	How to pick 1 primary + ≤ 2 secondary collaborators from <code>collaboration-map.md</code> , phase-filtered, and format the 4-level compression block
<code>dotnet-api-conventions</code>	auto (writing controllers/services in <code>src/</code>)	Provider abstraction rules, options pattern, the <i>real</i> error-handling pipeline (inline <code>app.Use</code> , not a middleware class)
<code>observability-net</code>	auto (logging/tracing work)	Serilog + OpenTelemetry wiring, correlation IDs, the <code>TryExtractUsage</code> 4-tuple cache-format gotcha
<code>pillars-audit</code>	<code>/deploy</code> (gate b), STEP 8/12	The 6-pillar pre-deploy audit (5 WAF pillars + Responsible AI), 34 codebase-specific GREEN/YELLOW/RED checks

2.3 Hooks & settings

File	Role	Behavior
<code>.claude/settings.json</code>	Project-level config — this is the closest thing to "appsettings.json at the .claude level" (there is no literal file by that name; this is its functional equivalent for the harness itself, as opposed to <code>src/lab-observability-api/appsettings.json</code> which configures the .NET app)	Registers one <code>PostToolUse</code> hook: after any <code>Write / Edit</code> on a <code>.md</code> file, run <code>markdownlint --fix</code> on it automatically. Also holds the permission allow/deny lists (denies reading secrets/ <code>.pfx</code> / <code>appsettings.Production.json</code> ; denies destructive git/az commands) and enables the Microsoft Docs MCP plugin.
<code>.claude/settings.local.json</code>	Machine-local permission overrides (not committed for secrets, but this one just allow-lists a few read-only Azure/PowerShell commands)	Merges on top of <code>settings.json</code>
<code>.claude/hooks/cert-tag.json</code>	A second hook definition, separate from <code>settings.json</code>	On every file touch, regex-matches the path against topic keywords (Telemetry, Resilience, Provider, Bicep, Cache, ...) and appends a cert-domain tag to <code>.claude/cert-tags-today.txt</code>

Finding: nothing in this repo currently reads `.claude/cert-tags-today.txt` — `/cert-update` derives touched domains purely from `01-summary.md`'s Certification Reinforcement section. The cert-tag hook's output is generated but unconsumed today.

2.4 Instructions — the master workflow

File	What's actually inside
<code>daily-workflow.md</code>	The one-page overview: the STEP 0–12 table (surface/model/what happens per step), four governing rules (chat only for STEP 2/4/5/11; Sonnet everywhere except STEP 5/11; close each chat step the moment it ends; STEP 0 is never optional), the "auto-print handoff" mechanic (each Claude Code step prints the next step's pre-filled prompt so you never hunt for it), an "if you get stuck" troubleshooting list, the command reference table (all 7 real commands + the 3 prompt-based gates), and a token-discipline reminder.
<code>daily-workflow-steps.md</code>	The long version — the literal copy-paste prompt text for every one of the 13 steps, including the full <code>/pitch</code> gate script (STEP 4, validates the CEO framing claim), the full <code>/devil</code> gate script (STEP 5, demands the strongest counterargument to the ADR before accepting it), and the full <code>/10x</code> gate script (STEP 11, scale-ceiling audit — optional Day 010–050, mandatory Day 051+). Also contains the exact STEP 8 pillars-audit prompt and the exact STEP 12 close-audit prompt referenced in §5 and §7 of this document.

You read the overview to orient; you paste from the steps file to actually run a step.

3. docs/standards/ — every file, what's actually in it

26 files. Each is read on-demand (none are in permanent hot context except `_principles.md`, `naming-conventions.md`, `azure-environment.md` per `CLAUDE.md`'s context-discipline rules). Grouped by what they govern, with a real content summary for each file — not just its name.

Posture & career

File	What's actually inside
<code>_principles.md</code>	"The North Star." The 5 traits of legendary architects (stay curious, understand people not just AI, do it right even when it's hard, explain at all 4 levels, build/build/build). The 5 daily posture-check questions answered at end of every day. The 3 phase-crossing signals (when you're really Phase 1 vs. 2 vs. 3, by behavior not by day number). The 3 working modes — Build/Study/Perform — with the posture and tooling proper to each.
<code>career-path.md</code>	The full 3-phase, 175+ day roadmap table (AI Engineer Days 1–20, Forward-Deployed Days 21–50, LLM Architect Days 51–100+), the cert-to-phase mapping table, the Phase 2/3 exit criteria checklists, the \$250k–\$500k+ compensation requirements (CEO framing from Day 9, real user validation, 3 public posts, 1 paying client), and the AI Paradigm Map (Generative AI / AI Agents / Agentic AI definitions, governance model per paradigm, layer mapping across gateway + all 3 future sites).
<code>competitive-intelligence.md</code>	A 2025–2026 snapshot of the AI engineering job market: the 6 things that separate \$300k–\$500k+ practitioners (eval-driven development, cost fluency, MCP/A2A protocol knowledge, owning failure modes, 90-second-CFO-pitch communication, security/compliance literacy), plus a 3-tier breakdown of what's now table-stakes vs. actively differentiating vs. emerging in the field.
<code>architect-thinking-template.md</code>	The required structure for every day's <code>03-architect-thinking.md</code> — the CEO Framing section format (a dollar amount or named risk, not "this improves efficiency"), and an explicit "what NOT to write" list (don't restate the file list, don't restate the completion checklist — those live elsewhere).
<code>graveyard.md</code>	A running, never-deleted table of every experiment that failed and what it taught — real entries from Day 5 through Day 9: the <code>az webapp deploy</code> TLS reset, the Azure naming collision that birthed the <code>-gio</code> suffix, the wrong Anthropic model ID that silently zeroed out prompt caching, the Day 8 incident where two separate chats authored <code>IBatchJobProvider</code> and <code>IBatchChatModelProvider</code> for the same feature.

Naming & environment

File	What's actually inside
<code>naming-conventions.md</code>	The <code>-gio</code> ownership-suffix rule (required for globally-unique Azure resources, optional for subscription-scoped ones), exact naming patterns with real examples for every resource type (App Service, Storage Account, Key Vault, Log Analytics, Action Groups, Budgets), and a changelog of when the convention itself changed and why.
<code>azure-environment.md</code>	The actual live environment: subscription ID, resource group name, the \$50/month budget and its 3 alert thresholds, every App Service application-setting key currently in use (<code>Anthropic__ApiKey</code> , <code>Anthropic__Model</code> , etc.), the real monitoring resource names and the live 5xx alert rule, and a "Platform Resource Roadmap" table mapping future Azure resources (SQL, Key Vault, AI Search, identity backend) to the specific future day each gets provisioned.

Build conventions

File	What's actually inside
testing-standard.md	What must be tested (every public method, happy path + failure path, no hard line-count target). The real integration-test pattern used in this repo: GatewayWebApplicationFactory substitutes IChatModelProvider / IBatchChatModelProvider with fakes at the seam — not an HTTP-level mock — corrected this session after discovering it described a FakeHttpMessageHandler that was never built.
commit-convention.md	<code>type(scope): subject</code> commit message format rules, a library of real example commit messages from this repo's own history, and the day-close commit sequence (what gets committed, in what order, at STEP 10/12).
error-handling-standard.md	The real ApiError 3-field contract (<code>code</code> / <code>message</code> / <code>correlationId</code> , camelCase, no <code>details</code> field). The actual error code taxonomy as emitted by the code today (<code>invalid_request</code> , <code>prompt_too_long</code> , <code>batch_size_exceeded</code> , <code>stream_error</code> , <code>claude_provider_error</code> , <code>internal_error</code>). The real ClaudeProviderException classification rules and the real retry policy per HTTP client. Rewritten this session — it previously described a fictional exception hierarchy and a middleware class that was never built.
api-versioning.md	URL-path versioning strategy (<code>/api/v{N}/</code> , no prefix until a v2 actually exists), a breaking-vs-non-breaking change checklist, and the deprecation policy for retiring an old version.
dependency-policy.md	A checklist for whether a new NuGet package should be added (maintained, used by Microsoft/major vendor, zero HIGH/CRITICAL vulnerabilities, MIT/Apache/BSD license), the update cadence table (security patches same day, minor versions every 2 weeks, major versions need an ADR), and how to document an exception when a vulnerable transitive dependency can't be updated.

Cost & performance

File	What's actually inside
cost-governance.md	Token budget rules, the requirement that prompt caching (<code>EnablePromptCaching = true</code>) is the default and required production state, and the batch-vs-real-time routing decision rules.
slo-performance.md	Latency target tables (p50/p95/p99) per endpoint with the actual OpenTelemetry span name each target is measured against (<code>ai.chat.complete</code> , <code>ai.provider.stream.ttft_ms</code>) — corrected this session after it cited two span names that were never built. Also the availability SLO and alert-rule definitions.
multi-turn-context.md	Explicitly forward-looking — labeled "Day 10 scope," not yet built. Covers the planned context-window token budget, the truncation strategy for long conversations, the conversation-history contract, and a SessionOptions design sketch (30-minute inactivity TTL).

Security & compliance

File	What's actually inside
security-standard.md	The real input-validation pattern (manual guard clauses in each controller action — not automatic <code>[ApiController]</code> model-state validation, corrected this session), the secrets-management rule (<code>dotnet user-secrets</code> locally, App Service env vars in Azure, never <code>appsettings.json</code>), an OWASP-style enforcement checklist, and the Phase 2 rate-limiting plan (not yet implemented).
responsible-ai.md	The content-filtering roadmap: what's enforced today (only a max-length check) versus Phase 2 plans (PII detection/redaction, prompt-injection detection, blocked-content categories) via a future IContentFilter interface.

Operations

File	What's actually inside
<code>monitoring-runbook.md</code>	The alert inventory (what's wired today), a daily monitoring checklist, and the P1–P4 incident-response procedure with severity definitions.
<code>kql-cookbook.md</code>	12 ready-to-run Kusto queries against the live Application Insights workspace — latency percentiles, the 10 slowest requests with correlation IDs, hourly error rate, cache hit rate, batch cost vs. sync-equivalent cost, and TTFT p50/p95/p99 for the streaming path.
<code>provider-onboarding.md</code>	The step-by-step playbook for adding a new LLM provider (Azure OpenAI, Bedrock, Foundry): which interfaces to implement, which files to create, the required documentation checklist, and the actual test pattern to follow (provider-seam fakes, not an HTTP-level fake — corrected this session).

Evaluation & future paradigms

File	What's actually inside
<code>eval-framework.md</code>	Why evaluation is an architectural concern (model/prompt/provider changes can silently change quality with no test catching it). The 6 evaluation dimensions — correctness, relevance, groundedness, safety, latency, cost — and which are measured today (latency + cost, via telemetry) versus Phase 2 (the other four, via LLM-as-judge). A Phase 2 architecture sketch: test dataset → gateway → response collector → judge.
<code>agent-patterns.md</code>	The governing standard for AI Agent and Agentic AI work. 3 implementation patterns (single-turn generative, single-agent tool use, multi-agent orchestration) each with concrete rules. A tool-design principles table (idempotent, typed schemas, observable, bounded). A failure-modes-and-guardrails table (infinite loops, tool call explosion, context exhaustion, hallucinated tool calls). MCP and A2A protocol notes. A cost-attribution table reconciled this session to use the real <code>ai.provider.*</code> / <code>ai.agent.*</code> telemetry prefix. An anti-patterns list (recursive self-calls, ad hoc handoffs, silent failure, unbounded loops).
<code>rag-patterns.md</code>	Phase 2 stub — explicitly designed, not built. RAG architecture sketch, chunking strategy notes, and the planned Azure AI Search integration.

People & communication

File	What's actually inside
<code>collaboration-map.md</code>	~20 collaborator roles (Security/AppSec, Product Manager, Backend Engineers, DevOps/SRE, Frontend, Data Engineers, Eng Manager, Vendor Support, Account Executive, Customer Success, Customer Sponsor, Domain Experts, End Users, Customer IT/Security, the Champion, Compliance/Legal, FinOps, Peer Architects, Procurement, VP Eng/CTO, External Auditors) × all 3 career phases. Every cell names their problem, your posture toward them, the one crucial question to ask, and the failure mode if you get it wrong. <code>/collab-lens</code> reads one row of this file per day.
<code>public-documentation.md</code>	The four-level writing rule (10-year-old / CEO / Engineer / Architect) required for anything that will appear on the public portfolio site, plus a 7-audience table (engineers, architects, hiring managers, security auditors, DevOps, junior developers, LinkedIn contacts) and each one's jargon tolerance.
<code>portfolio-strategy.md</code>	What makes this repo portfolio-worthy (ADRs with real tradeoffs, cost-conscious engineering, provider abstraction, observability-first design), the public website tech stack recommendation (VitePress — reads <code>.md</code> directly, zero-config syntax highlighting), and a LinkedIn/go-public checklist.

4. Other key files referenced throughout this document

File	What's actually inside
<code>CLAUDE.md</code> (repo root)	The project's own instructions file, loaded into every Claude Code session automatically. Owner & context, the full stack list, the naming-convention summary, the command list, the architecture top-level map, the full standards index (the table this document's §3 expands on), the repo conventions (day-folder naming, ADR rules, the backtick-formatting rule), the Provider Abstraction Contract (the exact methods <code>IChatModelProvider</code> must have), the Gotchas list (TLS resets, the Anthropic model-ID trap, the cache TTL trap), the full North Star/SaaS platform vision, and the context-discipline rules that decide what stays in hot context vs. what's read on demand.
<code>README.md</code> (repo root)	The public-facing entry point — a short project description and the same top-level folder map as <code>CLAUDE.md</code> , written for a human skimming GitHub rather than for Claude Code.
<code>docs/notes/_index.md</code>	One table, one row per day: day number, slug, track (Build/Hybrid), and status (Complete/In Progress). The single source of truth for "what day are we actually on" — <code>CLAUDE.md</code> deliberately has no day counter of its own.
<code>docs/notes/changelog.md</code>	The running file-change audit log — one <code>## Day NNN</code> section per day, each a flat table of every file touched that day with a Step label and a one-line Change description. Replaced the old one-file-per-day convention on 2026-06-16 specifically to stop cross-cutting fixes from being scattered across closed historical folders.
<code>docs/adr/ADR-NNN-*.md</code>	One Architecture Decision Record per significant decision (11 exist as of Day 9). Each has a required shape: Status, Date, Context, Decision, Consequences, and Alternatives Considered (at least one real alternative with a stated rejection reason). Never edited once Accepted — only superseded by a new ADR, or factually corrected for things like a renamed file path.
<code>.markdownlint.json</code> (repo root)	The lint rule overrides for every <code>.md</code> file in the repo: everything on by default, with 6 rules explicitly disabled — <code>MD013</code> (line length), <code>MD024</code> (duplicate headings), <code>MD025</code> (single top-level heading), <code>MD036</code> (emphasis used instead of a heading), <code>MD041</code> (first line must be a top-level heading), and <code>MD060</code> .
<code>.gitignore</code> / <code>.gitattributes</code>	<code>.gitignore</code> excludes build output (<code>bin/</code> , <code>obj/</code> , <code>publish/</code>), machine-local hook output (<code>.claude/cert-tags-today.txt</code>), and stray test artifacts. <code>.gitattributes</code> forces consistent CRLF line endings on Windows so git stops reporting line-ending-only diffs on every edit.

5. The daily workflow, step by step

Step	Surface	Model	What happens
0 CLOSE (prev)	Claude Code	Sonnet	Verify the previous day is fully frozen — audit, posture-check, changelog row all present — before starting anything new
1 PRUNE	Manual	—	Move the previous day's hot-context files out; rotate which ADRs stay loaded
2 DECIDE	Chat (new)	Sonnet	Lock the day's workload and slug
3 SCAFFOLD	Claude Code	Sonnet	<code>/new-day N slug</code> — build the empty skeleton
4 DRAFT	Chat (new)	Sonnet	Full 13-section Day N summary; <code>/pitch</code> gate validates the CEO framing
5 ADR	Chat (new)	latest Opus	Reason out the architectural decision (skip if none this day); <code>/devil</code> gate stress-tests it
6 POPULATE	Claude Code	Sonnet	Fill summary + checklist, run <code>/adr</code> , run <code>/collab-lens</code>
7 BUILD	Claude Code	Sonnet	Implement each phase of the day's plan
8 TEST + AUDIT	Claude Code	Sonnet	<code>dotnet test</code> , the 6-pillar audit, <code>symbol-drift-check.js</code> — all before deploy is allowed
9 DEPLOY	Claude Code	Sonnet	<code>/deploy</code> + post-deploy verification
10 DOCUMENT	Claude Code	Sonnet	Update docs, commit, <code>/cert-update</code>
11 REFLECT	Chat (new)	latest Opus	Posture check (5 questions); <code>/10x</code> gate audits the scale ceiling (mandatory Day 051+)
12 CLOSE	Claude Code	Sonnet	Save posture check, final commit, <code>/repo-audit</code>

Rule of thumb: Sonnet does all the building; the latest Opus is reserved for the two steps where reasoning depth — not execution — is the actual deliverable (ADR reasoning, posture reflection).

6. A day in the life — worked example

Tracing one real artifact, `IBatchChatModelProvider`, through the whole stack on the day it was built (Day 008):

STEP 2 (chat) — decide: "batch API for cost control" is the Day 008 workload

STEP 3 — `/new-day 8 batch-api-cost-controls` creates `docs/notes/Day-008/`, a `## Day 008` section in `changelog.md`, and `docs/architecture/day-008-batch-api-cost-controls.md`

STEP 5 (chat, latest Opus) — reasons out: new seam vs. extending `IChatModelProvider` → new seam wins (different computation model, not just different speed)

STEP 6 — `/adr introduce-parallel-batch-provider-abstraction` creates `ADR-010`, Status: Proposed

STEP 7 — builds `IBatchChatModelProvider.cs`, `ClaudeBatchChatModelProvider.cs`, `AiBatchController.cs`; the `dotnet-api-conventions` skill governs the shape

STEP 8 — `pillars-audit` finds two RED items (missing `MaxPromptLength` guard) → fixed → re-audit GREEN; `symbol-drift-check.js` confirms every new type is referenced correctly everywhere it's mentioned

STEP 9 — `/deploy`; the `azure-deploy` skill's Kudu path ships it to `app-ai-lab-api-dev-eastus-gio`

STEP 10 — `/cert-update 008` writes AI-102 and AZ-305 domain content citing the batch pattern; `changelog.md`'s Day 008 section gets every file logged

STEP 12 — `/repo-audit 8` confirms ADR-010 has all required headings, no drift, day folder complete

7. `scripts/` — automated guards

Script	Triggered by	Checks
<code>md-backtick-wrap.js</code>	<code>settings.json</code> hook (every <code>.md</code> write/edit), <code>/repo-audit</code> Check 4	Wraps bare file paths, slash commands, and HTTP paths in backticks per the markdown convention. Idempotent — re-running on clean content changes nothing.
<code>symbol-drift-check.js</code>	STEP 8, <code>/repo-audit</code> Check 9	Rebuilds the actual symbol/telemetry-name table from <code>src/</code> on every run; flags any backtick-wrapped doc reference that doesn't match current code — including symbols that were <i>documented but never built</i> . Also flags single-letter backtick spans as likely truncation/corruption (added 2026-06-16 after recovering 80 such occurrences).

8. Findings from this audit

Fixed — severe: a buggy `md-backtick-wrap.js` replace callback (before its fix in commit `1e33390`) had collapsed full paths/commands down to single letters inside backticks across 50 files, 80 occurrences — `/health` → ``h``, `/cert-update`

→ ``c``, etc. Recovered every instance from git history and added a permanent guard so this class of corruption can't recur silently.

Fixed: 6 source files (`ApiError.cs` , `TestController.cs` , `HttpContextExtensions.cs` , `CorrelationIdMiddleware.cs` , `ClaudeProviderException.cs` , the main `.csproj`) existed with zero changelog provenance — every `src/` file now traces to a Day NNN.

Open, not a bug: `.claude/hooks/cert-tag.json` generates tags nothing consumes (see §2.3). Worth deciding whether to wire it into `/cert-update` or remove it.

Open, accepted debt: ADR-001/002/003 predate the "Alternatives Considered" convention. Not fixed — adding that section retroactively to an Accepted ADR isn't a "factual correction" under this repo's own ADR-immutability rule.

9. Expected results

When every layer is working together correctly, a closed day means: the code builds and tests pass; the ADR (if any) has all required headings and no open placeholders; every touched file has a `changelog.md` row under that day's section; every cert domain marked Primary/Secondary in the day's summary has a corresponding day-mapping row; `symbol-drift-check.js` and `markdownlint` both report zero findings; the deploy succeeded and was verified against the live App Service; and the posture-check honestly names what's still YELLOW. `/repo-audit` is the single command that verifies all of this at once — a clean run is the actual definition of "Day N is done."